ForthScript

(FORTH 2x2)

A Biological Forth



by Jimmy Söderman

THE COMPLETE MANUAL · 16-BIT · 64K · INDIRECT THREADED · 1985

> *Every word knows its parent. A machine can raise a second one, word by word, until it stands on its own, and the cord can be taken back.*

How to use this manual

The system in this manual is **ITCH**: one HTML file (`ITCH-TINY.html`) that holds a 16-bit Forth machine, its chips, the language written in itself, and a second machine it can build. The manual is a companion to that file. It explains and indexes the file, and the file stays the only thing you need to run it.

Part	What it answers	Read it when
I · Module Dependency Map	What is in the file, what needs what, how to remove or add a piece safely	You want to trim, extend or port the system
II · Memory Map & Bootstrapping	Where everything lives on the 64K bus; how one machine boots, and how it builds a second	You want to understand the machine and the 2x2 design
III · Word Dictionary Reference	Every word on a booted host: stack effect, where it lives, what it does	You are writing Forth
IV · The Quick Reference	A terse, RetroForth-style card of the language	You want an answer in ten seconds

Why "biological"? Three things in ITCH behave like living things. Every word carries a **PARENT** cell, so it knows the word that made it (`SEE` shows the family line, `CHILDREN` lists the offspring, and `CLONE` makes one from another). **Motors** are small workers that live in the frame interrupt, are born with `ADDER` or `CODER`, and die with `KILL`. And the host can **reproduce**: it builds a second machine, byte for byte, then cuts the cord so the new machine lives on its own. `TETHER` takes the cord back.

Conventions. Stack effects are written `( before -- after )`. The top of the stack is on the right. `n` is a signed number, `u` unsigned, `f` a flag (0 false, -1 true), `a` an address, `c` a character, `c-addr` a counted string (a length byte followed by the characters), `xt` a word's address, and `"name"` means text read from the input. `C:` marks the compile stack and `R:` the return stack. Hex numbers are written `$F000`.

Contents

- **Part I · Module Dependency Map**: the file layer by layer · the machine · the ten modules · the dependency graph · removing, adding and trimming safely · what the target gets
- **Part II · Memory Map & Bootstrapping**: the 64K bus · registers · devices · what a word looks like · the 2x2 stacks · booting one machine · two machines · the monitor ROM · the bootstrap engine, step by step
- **Part III · Word Dictionary Reference**: all 493 host words by module · the target-mode words · the switched-off words · the trap messages
- **Part IV · The Quick Reference**: the language on a few screens

---

PART I · Module Dependency Map

1. The file, layer by layer

ITCH is one HTML file with four layers. Each layer is a set of `<script>` blocks, and each block has a banner comment in the file that says what it is and how to switch it off.

```
3  THE PAGE (JavaScript)   terminal-ui: log, prompt, RUN/STOP, the Worker   shows the machine
4  PROGRAMS (Forth)        demo-balloons steer logo clock paint piano tour  not part of the
                           cut                                              system
2  THE SYSTEM (Forth)      BOOT · NUMBERS · TOOLS · GRAPHICS · MOTORS ·     read at power-on,
                           TIME · SOUND · META · SERIAL · TETHER            in this order
1  THE MACHINE (JavaScript) KERNEL + chips: terminal video keyboard clock   the "hardware"
                           pointer sound uart  ·  MONITOR (the target's ROM)
```

Layer 1 is what a real board would give you in silicon and ROM. Layer 2 is the language, written in Forth and compiled at power-on by the kernel's seed interpreter. Layer 4 is ordinary Forth programs. Layer 3 is the display: it runs the machine, but the machine never calls it.

2. The machine: kernel and chips

Block	Section	Source	Primitives it adds	Registers on the bus
`vm-kernel`	1	38.5 KB	41 CODE words + the WORD/FIND vectors (stack, memory, inner interpreter, seed compiler)	$00–$3F, all RAM areas
`terminal-chip`	1a	0.4 KB	none	$F600 OUT-HEAD/TAIL, $F700 ring
`video-chip`	1b	4.1 KB	`CLS` `PLOT` `POINT` `SPLOT`	$F000 sprites, $24 FRAMES, $28 VBL vector
`keyboard-chip`	1c	1.2 KB	none	$F040 HEAD/TAIL, $F080 held keys, $F100 ring
`clock-chip`	1d	0.5 KB	none	$F060–$F066
`pointer-chip`	1e	0.3 KB	none	$F048 four points
`sound-chip`	1f	0.9 KB	none	$F200 four voices
`uart-chip`	1g	0.7 KB	none	$F300 heads/tails, $F400 TX, $F500 RX
`target-monitor`	1h	7.8 KB	none (it *is* ROM code)	runs only inside the target

Only the video chip adds primitives. Every other chip is **pure registers**: Forth talks to it with `C@ C! @ !`, as a 6502 talks to a 6551 ACIA or a keyboard buffer. That is why the system stays small and portable. A port to real hardware replaces each chip with the real device at the same addresses, and the Forth above it does not change.

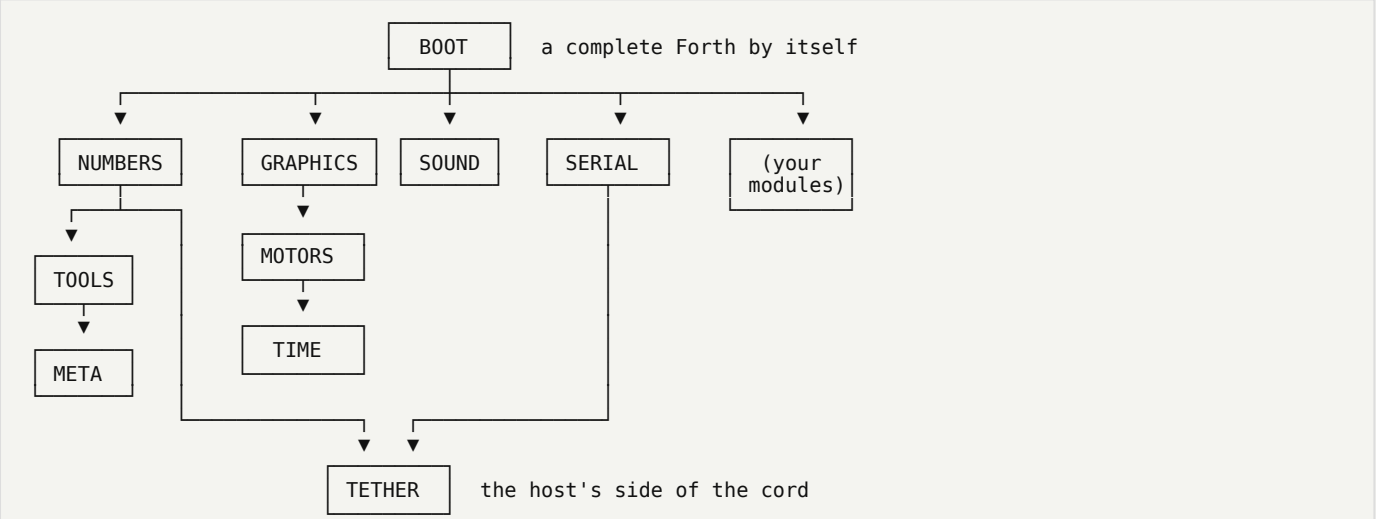
The kernel knows nothing about any chip. A chip that needs a power-on routine (the video chip, for its primitives; the monitor ROM, for its reset) adds it to the `DEVICES` list, and power-on calls those routines *after* the kernel's own primitives. So the CPU's native numbers never move, and two machines with the same chips give every primitive the same xt.

3. The system: ten Forth modules

Sizes are measured on a booted host. *Dictionary* is the RAM each module occupies after it is compiled.

#	Module	Source	Dictionary	Words	Needs (modules)	Needs (chips)	What it gives you
2	**BOOT**	35.5 KB	6,406 B	199	nothing	terminal, keyboard	the whole language: compiler, control flow, CASE, strings, WORD FIND NUMBER?, recognizers, INTERPRET, QUIT, errors, MARKER
2b	**NUMBERS**	0.9 KB	186 B	8	BOOT	—	`HEX DECIMAL . U.` number output (and `(.) (U.)`, digits alone)
2c	**TOOLS**	4.4 KB	1,499 B	24	NUMBERS	—	`SEE` (decompiler), `.S`, `CHILDREN`, lineage
2d	**GRAPHICS**	2.7 KB	622 B	22	BOOT	video, pointer	sprites, `LINE`, `VBL`, the pointer
2e	**MOTORS**	5.0 KB	1,528 B	49	GRAPHICS	video (VBL)	background workers run every frame
2f	**TIME**	1.9 KB	488 B	14	GRAPHICS, MOTORS	clock	`TIME&DATE MS EVERY LATER ALARM`
2g	**SOUND**	0.7 KB	279 B	10	BOOT	sound	`TONE QUIET PLAY`
2h	**META**	5.1 KB	1,472 B	24	NUMBERS, TOOLS	—	Forth running Forth: `META TRACE`
2i	**SERIAL**	2.0 KB	450 B	11	BOOT	uart	`SEND RECV RX-ACCEPT SERVE`
2j	**TETHER**	20.2 KB	4,666 B	87 (+ 22 target-mode)	NUMBERS, SERIAL	uart	the second machine: `TARGET … HOST`, metacompiling, `CUT TERM TETHER`
	total	78 KB	17,596 B	448			12.3 KB of the 30 KB dictionary left free

The dependency graph



Leaves (nothing depends on them, so each can be removed alone): **TIME, SOUND, META, TETHER. Trunks** (removing one breaks everything below it): **NUMBERS, TOOLS, GRAPHICS, MOTORS, SERIAL.**

Verified: what breaks when a module is missing

This was measured by booting the full system with one module left out:

Leave out	Result
NUMBERS	TOOLS fails: `UNKNOWN WORD: .`
TOOLS	META fails: `UNKNOWN WORD: .NAME`
GRAPHICS	MOTORS fails: `UNKNOWN WORD: FRAMES`
MOTORS	TIME fails: `UNKNOWN WORD: CODER`

Leave out	Result
SERIAL	TETHER fails: `UNKNOWN WORD: SEND`
TIME, SOUND, META or TETHER	boots cleanly

It was also verified the other way round: BOOT alone boots, and every module boots *and works* with only BOOT plus the modules in its *Needs* column.

4. How to remove, add or trim safely

The one switch: the BOOT list

The page reads the system from one list, near the top of the `terminal-ui` script:

```
const BOOT = ["forth-boot", "forth-numbers", "forth-tools", "forth-graphics", "forth-motors",
              "forth-time", "forth-sound", "forth-meta", "forth-serial", "forth-tether"];
```

To leave a module out, delete its name from this list. The module's text stays in the file, but nothing reads it. The test adapter (`clean_adapter.js`) keeps the same list, so change it there too if you run the tests.

Rules

1. **Remove leaves first.** Take out a module only after everything that needs it (the graph above).
2. **Keep the order.** A module may only use words defined above it. The list is already in a safe order; add new modules after the ones they need.
3. **BOOT is not optional.** It *is* the language. The kernel alone has only a seed interpreter, and BOOT sets the `QUIT-VECTOR`, without which the page refuses to start (`boot: the boot script never set the QUIT vector`).
4. **A module that hooks the system must unhook itself too.** MOTORS and TETHER chain cleanup onto `FORGETTING` so that `MARKER` stays safe. A new module that keeps pointers into the dictionary should do the same:

```
: FORGET-MINE ( a -- a ) [ ' FORGETTING >BODY @ , ]   \ run the hook chained before us
  ( drop whatever lives at or above a ) ;
' FORGET-MINE IS FORGETTING
```

1. **Demos don't count.** The `demo-*` blocks are programs, not system. Deleting one only removes it from the menu.

Ready-made trims

Goal	BOOT list	Loses
Minimal Forth	`forth-boot`	number output (`.`), `SEE`, graphics, everything optional
Headless target set (what the cut demo builds)	`boot numbers tools meta serial`	graphics, motors, time, sound, tether
Graphics, no cord	all but `forth-serial forth-tether`	the second machine
Everything	as shipped	—

Trimming inside a module

Some parts of BOOT are switched *off* in place and cost nothing until you switch them on:

- **Counted loops** (BOOT §13): `DO ?DO LOOP +LOOP I J LEAVE UNLOOP`. Delete the leading `\ ` of each line. `(DO) (LOOP) UNLOOP I DO LOOP` are the basic set, and the others each need only those. TOOLS has two matching lines for `SEE` (search for `(DO)`).
- **Speed primitives** (kernel APPENDIX): 20 Forth words in BOOT can become machine code: the 17 tagged `\ [PRIM]` (`OVER ROT NIP 2DUP 2DROP R@ 1+ 1- XOR 0= 0< = < U< UPC SKIP PARSE`) and `NEGATE INVERT OR` on BOOT's first lines, which cannot carry a tag. Uncomment the line in `speedPrimitives()` **and** delete the Forth definition. All of them together make the interpreter about 3× faster.

- **WORD and FIND** are deferred. `' (WORD) IS WORD ' (FIND) IS FIND` puts them back on the kernel's fast seed versions, with no JavaScript change.

Adding a chip

A chip is a script block. If it needs one, it pushes a power-on routine onto `DEVICES` (as the video chip does) and a per-frame routine onto `ON_FRAME` (today only the page's terminal uses that). Give it registers at a free address above `$F000`, write a small Forth module that names them (as SOUND does in 0.7 KB), and add both to the file. The rule is the one the others follow: **no primitives unless Forth truly cannot do it**. Only the video chip needs them, because its screen memory is not on the bus.

5. What the target machine gets

The second machine (the *target*) is powered on with the **kernel, the terminal chip, the UART and the monitor ROM**, and nothing else. It has no BOOT, no video, no keyboard. Everything else it gets, it gets over the cord from the host (Part II). The cut demo gives it `BOOT NUMBERS TOOLS META SERIAL`. GRAPHICS could not go there, because the target has no video chip and so no `PLOT`.

PART II · Memory Map & Bootstrapping

1. One 64K bus holds everything

ITCH is a 16-bit machine. Cells are 2 bytes, **little-endian** (the 6502's order), and addresses wrap at 64K. There are no hidden JavaScript structures: every register, stack, word and device register is a byte in RAM. That is what makes the machine portable, and what lets it stop between any two steps and carry on later.

```
$0000  +-----------------------+  the "zero page": SP RP IP HERE LATEST MSG …
$0040  | REGISTERS  (64 bytes) |
       +-----------------------+  a line being read (BOOT at power-on, SERVE's lines)
$0440  | TIB  terminal input buffer 1K |
       +-----------------------+  WORD leaves its counted name here
$04C0  | PAD  128 bytes        |
       +-----------------------+  depth in SP ($0C), bytes from the bottom
$05C0  | DATA STACK      256 B (128 c) |
       +-----------------------+  depth in RP ($0E)
$06C0  | RETURN STACK    256 B (128 c) |
       +-----------------------+  depth in CSP ($02); frames + IF/BEGIN holes
$07C0  | COMPILE STACK   256 B |
       +-----------------------+  depth in CXP ($2C); one mode per frame
$0800  | CONTEXT STACK    64 B |
       +-----------------------+  kernel words to KEND ($09CE), then BOOT, modules,
       | DICTIONARY  30K       |  your words. HERE is $0A.
       |   ↓ HERE grows up     |
$8000  +-----------------------+  the page copies your program here; QUIT reads it
       | LOADED TEXT  27K      |
$EC00  +-----------------------+  20 counted messages, "burned" at power-on
       | ROM: trap message table |
$EF00  +-----------------------+  a message built at run time (UNKNOWN WORD: …)
       | ERRBUF                |
$F000  +-----------------------+  sprites, keys, pointer, clock, voices, UART, terminal
       | I/O: the chips' registers |
$FFFF  +-----------------------+
```

The register page ($00–$3F)

Addr	Name	Holds	Forth word
$02	CSP	compile stack depth (bytes)	`CSP`
$04	>IN	how far into the source	`>IN`
$06	#TIB	length of the source	`#TIB`
$08	LATEST	newest header made	`LATEST`
$0A	HERE	next free dictionary byte (DP)	`DP`
$0C	SP	data stack depth (bytes)	`DEPTH` `SP!`
$0E	RP	return stack depth (bytes)	`RP@`
$10	BASE	number base	`BASE`
$12	IP	instruction pointer	—
$14	SPACE	where new definitions are built (0 = here, else a space table)	`CURRENT-SPACE`
$16	KEND	where the kernel's own words end	(`KERNEL-HEAD`)
$18	SRC	address of the text being read	`SRC`
$1A	MSG	current error message (counted), 0 = none	`MSG`
$1C	LINE	source line number	`LINE` (BOOT)
$1E	TXT	next line of the loaded text	`TXT`
$20	TXTEND	end of the loaded text	`TXTEND`
$22	STOP	halt register: non-zero stops NEXT	`HALT`
$24	FRAMES	frames shown so far (video chip)	`FRAMES`
$26	ME	the motor at work	`ME`
$28	VBL	the frame interrupt's vector	`VBL-VECTOR`
$2A	SELF	the data word whose DOES> code runs	`SELF`

Addr	Name	Holds	Forth word
$2C	CXP	context stack depth (bytes)	`CXP`
$2E	CFP	compile frame pointer (0 = no definition open)	`CFP`
$30	COMMON	newest header of the COMMON wordlist	`COMMON-WORDS`
$32	COMPILER	newest header of COMPILER	`COMPILER-WORDS`
$34	FORTH	newest header of FORTH	`FORTH-WORDS`
$36	CURRENT	the wordlist new words go into	`CURRENT`
$38	QUIT-VECTOR	the word entered to run text	`QUIT-VECTOR`
$3A	ABORT-VECTOR	the word entered to clean up	`ABORT-VECTOR`

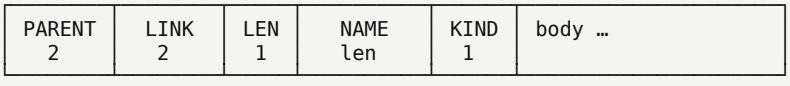
$00 and $3C–$3F are free.

The devices ($F000–$FFFF)

Addr	Chip	Registers
$F000	video	8 sprites × 4 cells: X Y PATTERN ON
$F040	keyboard	HEAD, TAIL (bytes) of the key ring
$F048	pointer	4 points × 3 cells: X Y DOWN (the mouse is point 0)
$F060	clock	SEC MIN HOUR DAY MONTH (bytes), $F066 YEAR (cell)
$F080	keyboard	held keys: a byte per key code, 255 while down
$F100	keyboard	the key ring, 256 bytes
$F200	sound	4 voices × 8 bytes: FREQ(cell) VOL WAVE ADDR(cell) LEN(cell)
$F300	UART	TX-HEAD TX-TAIL RX-HEAD RX-TAIL (bytes)
$F400 / $F500	UART	TX ring / RX ring, 256 bytes each
$F600	terminal	OUT-HEAD OUT-TAIL (bytes)
$F700	terminal	the output ring, 256 characters

Rings. The keyboard, the UART and the terminal pass bytes the same way: two byte registers, HEAD and TAIL, and a 256-byte ring. The writer puts a byte at TAIL and steps it; the reader takes at HEAD and steps it. HEAD = TAIL means empty; TAIL one behind HEAD means full. Because the registers are bytes, they wrap by themselves. A Forth reader that finds the ring empty, or a writer that finds it full, executes `WAIT` and sleeps until the next frame.

2. What a word looks like



- **PARENT** sits just *before* the xt: the word that made this one (0 = the machine itself). This is the biological part. `IS-A?` walks the family line, `SEE` prints it (`made by CONSTANT`), `CHILDREN` lists the offspring, and `CLONE` makes a new word from an old one.
- **LINK** is the previous word in the same wordlist.
- **LEN NAME** is exactly what `WORD` leaves in PAD. Bit 7 of LEN is free.
- **KIND** says how `EXECUTE` runs the body:

KIND	Name	Body	Run by
0	CODE	one byte: the primitive's native number	the native routine
1	COLON	a thread: xt xt xt … EXIT (`LIT n` and `BRANCH a` carry an inline cell)	DOCOL: push IP, IP := body

KIND	Name	Body	Run by
2	CREATED	ACTION (2 bytes: DOES> code or 0), then data	DODOES: push the data address; if ACTION ≠ 0, enter it with SELF := the word

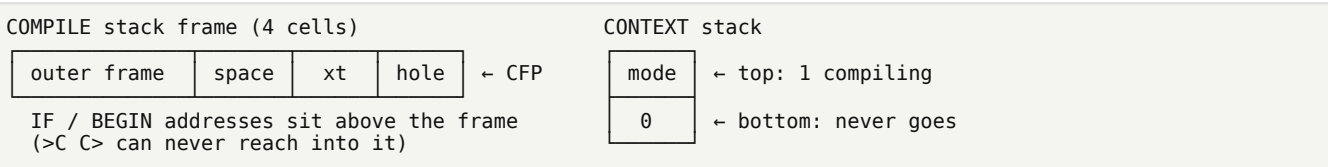
So there are only two ways to run a Forth word. VARIABLE, CONSTANT, DEFER and your own defining words all run through DODOES.

Wordlists decide behaviour, not flags. No header has an "immediate" bit. A word acts while compiling because it lives in **COMPILER** (`IMMEDIATE` moves it there), or acts the same in every mode because it lives in **COMMON** (`ALWAYS`). Everything else is in **FORTH**.

3. The 2x2: four stacks, one question each

	being built / worked on	navigating what already exists
execution	DATA values $04C0, depth SP	RETURN where we are in running threads $05C0, depth RP
compilation	COMPILE what is being built, where, and the holes $06C0, depth CSP	CONTEXT how to treat tokens now 0 interpret · 1 compile · 2 DOES> · 3/4 same, target

There is no STATE. A definition opens a frame on **both** compilation stacks, and `;` closes both:

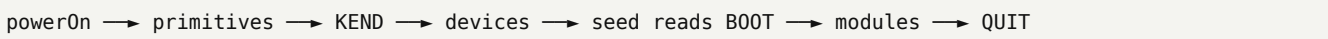


- **space**: where the word is built. 0 means this machine's RAM; otherwise it is the address of a **space table** of four xts: `HERE , PATCH OPEN`. The primitives `HERE`, `,` and `PATCH` are *vectored*. When the top frame's space is not 0, they jump through its table, like the C64 KERNAL's RAM vectors. **This one mechanism is what lets the same `IF THEN BEGIN LITERAL` compile into another machine.**
- **hole**: for a nested definition (a `:` inside a `:`), the outer word gets `BRANCH <hole>`, and the inner `;` fills the hole so the outer thread jumps over the inner word. This works at any depth.
- `[` pushes an interpreting frame on the context stack only, and `]` pops it. The word being built waits underneath.

Context frames and recognizers. The top context's *mode* picks a table of `(recognizer, action)` pairs. `INTERPRET` hands each token to the pairs in order, and the first recognizer that knows the token hands it to its action.

Mode	Frame (host only)	Frame (with TETHER loaded)
0 interpreting	COMMON run · FORTH run · "string" push · number push	target-mode prompt words · COMMON · (target mode) anything the target has runs *there* · FORTH · other target words run there · string · number
1 compiling, 2 DOES>	COMMON run · COMPILER run · FORTH compile · string/number literal	+ a target word becomes a REMOTE call
3 target compiling, 4 target DOES>	—	transition words · COMMON · COMPILER (their `HERE , PATCH` go to the target) · target-immediate runs *there* · target words and kernel words compiled · string (TSLITERAL) · number (LITERAL)

4. Booting one machine, step by step

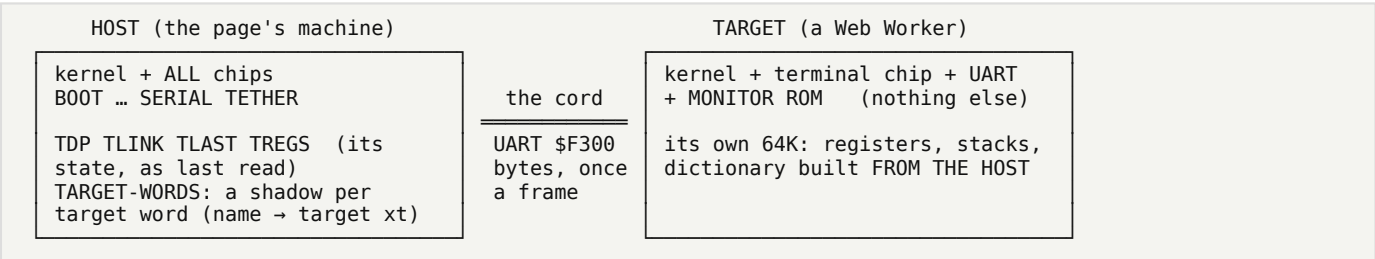


1. **Power on.** 64K of RAM is cleared. The ROM trap table is written at $EC00. HERE := $0800, BASE := 10, CXP := 2 (the bottom context frame, *interpreting*), CURRENT := FORTH.
2. **Primitives.** The kernel lays 41 CODE words (stack, memory, `LIT EXIT BRANCH 0BRANCH`, the seed `: ;`, `>CS OPEN? [ ] >C C> HERE , PATCH (CLOSE)` …). It also lays two tiny threads by hand (`@ EXECUTE EXIT` for vectors, and DOES>'s way back) and two vectors, `WORD` and `FIND`, aimed at the seed primitives `(WORD)` and `(FIND)`.
3. **KEND := HERE** ($09CE). Everything below is the CPU's own words, identical on every machine.
4. **Devices.** Each chip's power-on routine runs. The video chip adds `CLS PLOT POINT SPLOT`, after KEND, so the CPU's native numbers never move.
5. **The seed interpreter reads BOOT**, line by line through the TIB. It is a fixed JavaScript copy of the context frames: interpreting runs COMMON and FORTH words and pushes numbers; compiling runs COMMON and COMPILER words, compiles FORTH words and makes numbers literals.
    - *Stage 0* (11 lines, no comments possible yet): `LATEST >IN #TIB NEGATE INVERT OR (RELINK) IMMEDIATE ALWAYS \ (`.
    - §1–§17: stack words, arithmetic, control flow, comparison, headers, `: ;`, defining words, lineage, CASE, the terminal (`EMIT` in Forth), strings, keyboard, wordlists.
    - §18: `DEFER`. The kernel's `WORD`/`FIND` vectors are re-parented to DEFER.
    - §19: **Forth's own WORD and FIND** replace the seeds with `IS`.
    - §20–§22: number input, errors, and the **outer interpreter** with recognizers.
    - §23: **QUIT**, then `' QUIT QUIT-VECTOR ! ' ABORT ABORT-VECTOR !`. The system now has an entry point.
    - §24: MARKER and ANEW.
6. **Modules** follow in BOOT-list order, each read the same way.
7. **Check.** If QUIT-VECTOR is still 0, the boot fails: `boot: the boot script never set the QUIT vector`.
8. **Run.** For every program or typed line, the page copies the text to $8000, sets TXT/TXTEND and enters the word in QUIT-VECTOR. From here on it is all Forth: `QUIT → REFILL → INTERPRET → RECOGNIZE`.

Time slices. All machine state is in RAM, so the page runs the machine a slice of steps per frame. `WAIT` (the 65C02's WAI) sleeps until the frame interrupt. Each frame, the terminal takes what EMIT put in its ring, the video chip adds 1 to FRAMES and runs `VBL`, and the motors run.

Errors don't unwind. A primitive that cannot go on raises a numbered **trap**, which points MSG at message n in ROM. A Forth `ERROR` sets MSG and `HALT`s. The stacks stay exactly as they were for you to inspect; the next `QUIT` sees MSG and cleans up first.

5. Two machines: the 2x2 host/target design



Why it works: same power-on, same xts. Both machines run the same `powerOn`, so every primitive has the same xt on both. `' DUP` on the host *is* the target's DUP. The host keeps these kernel words in a wordlist of their own, **KERNEL-WORDS** (the FORTH headers below KEND), so they are always found, even when a host word shadows the name. For example, the motors' field `>C` shadows the compile stack's `>C`.

The monitor ROM: five commands and two signals

The target starts in its ROM monitor, which obeys the host one command at a time:

Byte	Command	Arguments	Reply
`W` (87)	write	addr(2) len(1) data…	none
`R` (82)	read	addr(2) len(1)	the len bytes
`X` (88)	execute	xt(2)	`O c` per character printed … then `K 0`, or `K 1 n msg` on error
`G` (71)	go	xt(2)	none: the word owns the UART from now on (**CUT**)
EOT (4)	*while serving*	—	none: back to the monitor, RAM untouched (**TETHER**)
`Y` (89)	sync	nonce(2)	`EOT Y nonce`: the handshake marker
ctrl-C (3)	*while serving*	—	BREAK: the served word stops and restarts

Every command fits the 256-byte RX ring (at most 244 bytes), and replies start only when they fit TX. That is hardware-style flow control, with no clocks read: the same bytes in give the same bytes out on any computer. The target runs 200,000 steps per heartbeat, about 60 heartbeats a second (plus one per message from the host).

The host's cord words, layer by layer

```
SEND RECV                       one byte                          (SERIAL)
SEND16 RECV16 CMD               a cell; a command
TC! T! TC@ T@ TMOVE TREAD       target memory
TCALL >T T> .TS                 run a word there; its stack
TRUN                            run a word there AS IF IT WERE OURS: our stack and the rest of
                                our line go there; its stack, >IN, HERE and new words come back
TARGET … HOST                   build there
CUT  TERM  TETHER               let go; talk to it; take it back
```

6. The bootstrap engine, rolled out

This is what happens when the cut demo runs `TARGET`, then BOOT NUMBERS TOOLS META SERIAL, then `HOST CUT`.

Step 1 · `TARGET`. `CONNECT` reads the target's registers the first time (`(PULL)`: one `R $04 52`) and keeps its HERE as **TDP** and its FORTH head as **TLINK**. Then CURRENT-SPACE := the **TARGET-SPACE** table (`THERE T, T! T-OPEN`). From now on `TARGET?` is true, so the tethered context frames (installed in MODE-FRAMES when the TETHER module loaded) send the prompt's words and every target word to the target.

Step 2 · a colon definition, e.g. `: LATEST 8 ;`.

1. `:` is the host's own (COMMON). Its `(OPEN)` sees a non-zero space and calls the table's OPEN, which is **T-OPEN**.
2. T-OPEN builds a header **at THERE, in the target** (`T-HEADER`: PARENT = the target's namesake of `:`, LINK, name, KIND 1) and a **shadow** here in TARGET-WORDS (name → target xt). It opens the frames on the *host's* compile and context stacks, with space = TARGET-SPACE and **mode 3**.
3. `8` is recognized as a number, so `LITERAL` compiles `LIT 8`. LITERAL says `,`, and `,` is vectored, so the cells land in the target (`T,`: one `W` command, no reply).
4. `;` is the TARGET-COMPILER's own. It lays `EXIT` there, closes both host frames (`(CLOSE)`), links the word into the target's FORTH list (`T-LINK`) and writes the target's HERE.

Step 3 · control flow needs no rewrite. `IF THEN BEGIN WHILE REPEAT CASE LITERAL RECURSE` are the *host's* BOOT words. They only ever say `HERE , PATCH`, and those follow the frame's space, so they compile target code unchanged.

Step 4 · words that must run on the target. Some words must *act* during the build. At the prompt, `CONSTANT DEFER IS` are TARGET-INTERPRETER words that run the target's own word of that name (`TRUN-NAMED`); `IMMEDIATE ALWAYS` do the same and then mark the shadow. Any other target word at the prompt is found by the `REC-THERE → TRUN` pair, and a target word that acts while compiling, inside a target definition, by `REC-TIMMEDIATE → TRUN`. **TRUN** ships the host's stack into the target's stack and the rest of the host's line into the target's TIB, sets the target's HERE, runs the word with `X`, then reads back its registers and stack. That is three round trips. Words the target made during the call get shadows (`CATCH-UP`), and words it forgot lose theirs (`PRUNE`).

Step 5 · transition words. BOOT's `S" ." ABORT" CREATE DOES> POSTPONE OF ; [']` would lay *host* xts. Mode 3 has its own versions (TARGET-COMPILER) that lay the **target's own** words, found by name when they run: `T-COMPILE (S")`. Until BOOT has defined `(S")` on the target, using `S"` there says `(S") is not on the target yet`. The build is self-consistent: it can only use what already exists there.

Step 6 · byte for byte. After BOOT NUMBERS TOOLS META SERIAL, the target's dictionary is **byte-identical** to the one a headless machine builds by reading the same text itself. The only difference is the value left in `DELIM`. `test_metacompile` checks this for BOOT NUMBERS TOOLS META (and with SERIAL added, both machines end at HERE $30CC).

Step 7 · `CUT`. `0 12 T!` empties its stack, then `G SERVE`. The monitor leaves the ROM and the target runs **SERVE**, a QUIT whose lines come from the UART (`RX-ACCEPT`, echoed). An error HALTs it, and the monitor re-enters SERVE like a reset vector. SERVE then prints `? message` and cleans up. The host can now be switched off: the target lives on.

Step 8 · `TERM`. The host becomes the target's terminal: bytes up the cord are shown, and your keys go down it. ESC leaves TERM.

Step 9 · `TETHER`. First the handshake, **SYNC**. The host sends **EOT**: the target's monitor takes the UART back, the served word is frozen, its unread input and unsent output are dropped, and its RAM is untouched. Then the host sends `Y` with a **nonce**, new each time (a count of syncs and the frame count), and the monitor answers `EOT Y nonce`. The served word may have printed anything, and some of it may still be on its way. So the host slides every byte coming up the cord through a 4-byte window until the window holds exactly that answer; everything before it was the served word's. No byte sent before the nonce existed can hold it. The wire overruns as a UART does (the host's side buffers 4K; a byte that finds it full is lost), so the answer itself may be lost: after 30 quiet frames SYNC asks again with a new nonce, and a late answer to the old one no longer matches. Then the host reads the target's registers and reconciles the shadows:

```
SURVIVOR   our newest shadow that is still true there (below its HERE, same name at its xt)
           → everything below it is agreed
PRUNE      shadows above it go         (words it forgot while we were away)
CATCH-UP   its headers above it get new shadows  (words typed to it in TERM)
```

A host that was switched off in between has no shadows at all, so it learns every word above the kernel, with one round trip per header (about 270 frames, 4.5 s). After `TETHER`, `TARGET`, `TRUN` and `CUT` work as before, even if the target was still printing when you typed `TETHER`.

Memory on both sides after a metacompile

	Host	**Target**
$0800–KEND	kernel words	kernel words (**same xts**)
above KEND	video prims, BOOT … TETHER (17.6 KB), then a shadow per target word	BOOT NUMBERS TOOLS META SERIAL (~10 KB), built over the cord
target state	TDP TLINK TLAST, TREGS (its $04–$37), TBUF	its own registers
who knows names	TARGET-WORDS (shadows), KERNEL-WORDS	its own COMMON COMPILER FORTH lists

---

PART III · Word Dictionary Reference

Every word on a fully booted host: **493 words**, in the order they are defined, grouped by where they live. The table was generated from the running system (each word's wordlist and kind) and the source (its stack comment). Descriptions were written from the definitions.

The flags column

Flag	Meaning
I	*Immediate*: lives in the COMPILER wordlist, so it acts while compiling
A	*Always*: lives in COMMON, so it acts the same whether interpreting or compiling
P	A `[PRIM]` candidate: a Forth word the kernel APPENDIX can replace with machine code
(blank)	an ordinary FORTH word: runs when interpreting, is compiled when compiling

A word defined twice appears twice. The later one shadows the earlier for new code, and old code keeps what it was compiled with. Watch `LINE`: BOOT's is the line-number register, and GRAPHICS' draws a line.

Kernel primitives (the CPU)

Word	Stack		What it does
`+`	`( a b -- a+b )`		Add.
`-`	`( a b -- a-b )`		Subtract.
`*`	`( a b -- a*b )`		Multiply (low 16 bits).
`/`	`( a b -- q )`		Divide, truncating toward zero.
`DUP`	`( x -- x x )`		Copy the top of the stack.
`DROP`	`( x -- )`		Discard the top of the stack.
`SWAP`	`( a b -- b a )`		Exchange the top two cells.
`@`	`( a -- x )`		Fetch the cell at a (signed 16-bit).
`!`	`( x a -- )`		Store x at a.
`C@`	`( a -- c )`		Fetch the byte at a.
`C!`	`( c a -- )`		Store the low byte of c at a.
`,`	`( x -- )`		Compile a cell at HERE. Vectored: builds in the open definition's space (here, or the target).
`AND`	`( a b -- a&b )`		Bitwise AND.
`MOVE`	`( from to n -- )`		Copy n bytes.
`UM/MOD`	`( lo hi u -- rem q )`		Divide the unsigned 32-bit number hi:lo by u. Traps on overflow.
`WAIT`	`( -- )`		Sleep until the next frame interrupt (the 65C02's WAI).
`(WORD)`	`( c -- c-addr )`		Seed WORD: parse a name up to delimiter c into PAD, upper-cased.
`(FIND)`	`( c-addr -- xt 1 | xt -1 | c-addr 0 )`		Seed FIND: exact-match lookup in COMMON, COMPILER, FORTH.
`EXECUTE`	`( xt -- )`		Run the word xt.
`>R`	`( x -- ) ( R: -- x )`		Move a cell to the return stack.
`R>`	`( -- x ) ( R: x -- )`		Move a cell from the return stack.
`RP!`	`( n -- )`		Set the return stack depth (bytes).
`LIT`	`( -- x )`		Push the inline cell that follows in the thread, and skip it.
`EXIT`	`( -- )`		Return from the current colon word.
`BRANCH`	`( -- )`		Jump to the inline address.
`0BRANCH`	`( f -- )`		Jump to the inline address if f is 0.
`'`	`( "name" -- xt )`		The xt of the next word in the input. Traps if not found.

Word	Stack		What it does
[']	("name" --)	I	Compiling: compile the next word's xt as a literal.
:	("name" --)		Seed colon: open a definition. BOOT's : replaces it.
;	(--)	I	Lay EXIT and close the open definition on both compilation stacks.
>CS	(space xt hole --)		Open a definition frame on the compile stack (and COMPILING on the context stack).
OPEN?	(-- f)		Is any definition open?
COMPILING?	(-- f)		Is the top context frame a compiling one?
[	(--)	I	Push an INTERPRETING frame: interpret inside a definition.
]	(--)		Pop the INTERPRETING frame [pushed: back to compiling.
(UNSELF)	(--)		Where DOES> code returns: restore SELF, then return.
>C	(x --) (C: -- x)		Push onto the compile stack (never into a frame).
C>	(-- x) (C: x --)		Pop the compile stack (never out of a frame).
HERE	(-- a)		Where the next compiled cell goes. Vectored, like , .
PATCH	(x a --)		Fill a hole left in a definition. Vectored, like , .
(CLOSE)	(--)		Close the open frame without laying EXIT (another space's ; uses it).
WORD	(c -- c-addr)		Deferred. Parse the next name up to delimiter c into PAD, upper-cased.
FIND	(c-addr -- c-addr 0 | xt 1 | xt -1)		Deferred. Look a name up: 1 = acts while compiling, -1 = FORTH word.

Video chip primitives

Word	Stack		What it does
CLS	(--)		Clear the screen to black.
PLOT	(x y rgb --)		Set a pixel; colour is $RGB.
POINT	(x y -- rgb)		Read a pixel back.
SPLOT	(x y rgb pattern --)		Set a pixel of sprite pattern 0-7 (colour 0 is transparent).

BOOT

Word	Stack		What it does
LATEST	(-- a)		Register $08: the newest header made.
>IN	(-- a)		Register $04: how far into the source.
#TIB	(-- a)		Register $06: length of the source.
NEGATE	(n -- -n)	P	Negate.
INVERT	(x -- ~x)	P	Flip every bit.
OR	(a b -- a|b)	P	Bitwise OR, from INVERT and AND.
(RELINK)	(wl --)		Move the newest word from CURRENT into wordlist wl.
IMMEDIATE	(--)		Move the newest word into COMPILER: it acts while compiling.
ALWAYS	(--)		Move the newest word into COMMON: it acts the same in every mode.
\	(--)	A	Comment to the end of the line.
(	("ccc)" --)	A	Comment up to the next).
OVER	(a b -- a b a)	P	Copy the second cell to the top.
ROT	(a b c -- b c a)	P	Rotate the third cell to the top.
NIP	(a b -- b)	P	Drop the second cell.
2DUP	(a b -- a b a b)	P	Copy the top two cells.
2DROP	(a b --)	P	Drop the top two cells.

Word	Stack		What it does
R@	(-- x)	P	Copy the top of the return stack.
RP@	(-- n)		Return stack depth in bytes.
DEPTH	(-- n)		Data stack depth in cells (SP counts bytes)
1+	(n -- n)	P	Add one.
1-	(n -- n)	P	Subtract one.
MOD	(a b -- r)		Remainder of a / b.
XOR	(a b -- x)	P	Bitwise exclusive OR.
+!	(n a --)		Add n to the cell at a
DP	(-- a)		Dictionary pointer: the next free byte
COMMON-WORDS	(-- a)		$30 the COMMON wordlist: its newest header
COMPILER-WORDS	(-- a)		$32 the COMPILER wordlist
FORTH-WORDS	(-- a)		$34 the FORTH wordlist
CURRENT	(-- a)		$36 the wordlist new words go into: FORTH-WORDS
TIB	(-- a)		Terminal input buffer: the boot script is read through it
PAD	(-- a)		Scratch: WORD leaves its counted string here
SRC	(-- a)		Address of the text being read: a line, the TIB, an EVALUATE string
BASE	(-- a)		Number base, for input and output
ALLOT	(n --)		Reserve n bytes of dictionary
C,	(c --)		Compile one byte into this machine's dictionary.
CURRENT-SPACE	(-- a)		$14 where new definitions are built: 0 here, or a space table
CSWAP	(C: a b -- b a)		Swap the top two compile-stack entries.
>MARK	(C: -- orig)		A hole, to fill later
>RESOLVE	(C: orig --)		Fill the newest hole with HERE
IF	(C: -- orig)	I	Compile a forward jump taken when the flag is 0.
THEN	(C: orig --)	I	End an IF (or ELSE): fill its hole with HERE.
ELSE	(C: orig -- orig')	I	End the IF part; start the part run when the flag was 0.
BEGIN	(C: -- dest)	I	Start a loop: remember where it begins.
UNTIL	(C: dest --)	I	Loop back to BEGIN while the flag is 0.
AGAIN	(C: dest --)	I	Loop back to BEGIN forever.
WHILE	(C: dest -- orig dest)	I	Inside BEGIN … REPEAT: leave the loop when the flag is 0. Two WHILEs may share one loop (close the second with THEN).
REPEAT	(C: orig dest --)	I	Close BEGIN … WHILE: jump back to BEGIN; WHILE exits land here.
0=	(n -- f)	P	True if n is zero.
0<>	(n -- f)		True if n is not zero.
0<	(n -- f)	P	True if n is negative.
0>	(n -- f)		True if n is positive.
=	(a b -- f)	P	True if a equals b.
<>	(a b -- f)		True if a and b differ.
<	(a b -- f)	P	Signed less-than.
>	(a b -- f)		Signed greater-than.
<=	(a b -- f)		Signed less-or-equal.
>=	(a b -- f)		Signed greater-or-equal.

Word	Stack		What it does
U<	(u u -- f)	P	Unsigned less-than.
U>	(u u -- f)		Unsigned greater-than.
WITHIN	(n lo hi -- f)		Lo <= n < hi, with one unsigned compare
?DUP	(x -- x x | 0)		Duplicate x if it is not zero.
MIN	(a b -- n)		The smaller (signed).
MAX	(a b -- n)		The larger (signed).
ABS	(n -- u)		Absolute value.
(HEADER)	(parent kind c-addr --)		Build a header at HERE: PARENT, LINK into CURRENT, name, KIND. Every defining word uses it.
HEADER	(parent kind "name" --)		Build a header for the next name in the input.
>CFA	(xt -- a)		Skip LINK LEN NAME: the address of the KIND byte
>ACTION	(xt -- a)		A CREATEd word's first body cell: its DOES> code, or 0
PARENT	(xt -- xt')		The word that made it (0: the machine itself)
SELF	(-- xt)		The data word whose DOES> code is running (0: none)
CSP	(-- a)		Compile stack depth, in bytes
CS	(-- a)		The compile stack, $06C0
CFP	(-- a)		Where the open definition's frame ends (0: none)
CXP	(-- a)		Context stack depth, in bytes
CX	(-- a)		The context stack, $07C0
BUILDING	(-- xt)		The word being compiled
MODE	(-- n)		The top context frame's mode
MODE!	(n --)		Set the top context frame's mode.
IN-DOES?	(-- f)		Are we compiling DOES> code?
(OPEN)	(parent kind c-addr --)		Build a colon header and open its frames; in another space, that space's OPEN does it.
:	("name" --)	A	Begin a colon definition. ALWAYS, so a : inside a definition builds a nested word.
:NONAME	(-- xt)	A	Begin a nameless colon definition; its ; leaves the xt.
LITERAL	(n --)	I	Compile "push n"
RECURSE	(--)	I	Compile a call to the word being built
(CREATE)	(parent "name" --)		KIND 2, ACTION 0: running it pushes its data address
CREATE	("name" --)	A	Make a data word: running it pushes its data address. Records its maker as PARENT.
(DOES>)	(--)		Run-time of DOES>: make the rest of the defining word the newest word's action.
DOES>	(--)	I	In a defining word: the rest is the action of the words it makes.
CONSTANT	(n "name" --)		Make a word that pushes n.
VARIABLE	("name" --)		Make a data word holding one cell, 0.
IS-A?	(xt ancestor -- f)		Is ancestor on xt's family line (xt itself, its PARENT, its PARENT's …)?
IN-LIST?	(xt wl -- f)		Is xt in wordlist wl?
IMMEDIATE?	(xt -- f)		Does xt act while compiling (COMMON or COMPILER)?
POSTPONE	("name" --)	I	Compile the next word's compile-time behaviour into the word being built.
CASE	(C: -- 0)	I	Begin a CASE … ENDCASE selection.

Word	Stack		What it does
OF	(C: -- orig)	I	In CASE: if x equals the value, drop x and run this branch.
ENDOF	(C: orig -- orig')	I	End an OF branch: jump to ENDCASE.
ENDCASE	(C: 0 orig… --)	I	End a CASE: drop x, fill every ENDOF's jump.
OUT-HEAD	(-- a)		$F600 a byte: where the terminal takes the next one
OUT-TAIL	(-- a)		$F601 a byte: where EMIT puts the next one
OUT-RING	(-- a)		$F700 256 characters on their way out
EMIT	(c --)		Print a character (via the terminal chip's ring; waits a frame when full).
PAGE	(--)		Clear the terminal (a form feed).
>BODY	(xt -- a)		A word's data field address.
CR	(--)		Start a new line.
SPACE	(--)		Print one space.
COUNT	(c-addr -- a n)		Counted string → address and length.
/STRING	(a n k -- a+k n-k)		Step k characters into a string.
TYPE	(a n --)		Print n characters from a.
FILL	(a n c --)		Fill n bytes at a with c.
KEY-HEAD	(-- a)		$F040 a byte: where KEY takes the next key
KEY-TAIL	(-- a)		$F041 a byte: where the chip puts the next key
KEY-HELD	(-- a)		$F080 a byte per key code: 255 while it is held
KEY-RING	(-- a)		$F100 256 bytes of keys waiting
KEY?	(-- f)		Is a key waiting?
KEY	(-- c)		Wait for a key and take it.
KEY-DOWN?	(c -- f)		Is that key held down right now? (for games)
KEYS	(a n --)		Queue n characters as if typed on the keyboard.
ACCEPT	(a n -- n')		Read a line of up to n keys into a, echoing; backspace edits. Leaves the count.
DELIM	(-- a)		The delimiter SKIP and PARSE are working with.
MORE?	(-- n)		Characters left: non-zero counts as true
PEEK	(-- c)		The character at >IN
>IN+	(--)		Step >IN one character on.
SKIP	(c --)	P	Step >IN over delimiter characters c.
PARSE	(c -- a n)	P	Take the text up to delimiter c; step past it.
S,	(a n --)		Lay down a counted string
(S")	(-- a n)		Run-time of an inline string: push it, continue after it.
SLITERAL	(a n --)	I	Compile "push this string": (S") and the string inline
S"	("ccc<quote>" -- a n)	A	A string up to the next ": compiled into a definition, or pointing into the source at the prompt.
."	("ccc<quote>" --)	A	Print the text up to the next " (compiled into a definition, or now).
.(	("ccc<paren>" --)	A	Print the text up to) now, even inside a definition.
NAME>STRING	(xt -- a n)		A header's name (bit 7 is free)
UPC	(c -- C)	P	Upper-case one character.
NOCASE=	(c1 c2 -- f)		Are two characters equal, ignoring case?
UPPER	(a n --)		Upper-case n characters in place.
S=	(a1 a2 n -- f)		The same n characters, in any case?

Word	Stack		What it does
WANT	(-- a)		The name length FIND-IN is looking for.
FIND-IN	(a n wl -- xt|0)		Look a name up in one wordlist.
FIND-NAME	(a n -- xt|0)		Look a name up in COMMON, COMPILER, FORTH.
VISITOR	(-- a)		The word EACH-WORD shows each header to.
(VISIT)	(wl --)		Show every header of one wordlist to VISITOR.
EACH-WORD	(xt --)		Run xt (x --) on every header, list by list, newest first.
SOUGHT	(-- a)		What a visitor is looking for.
(.WORD)	(x --)		WORDS's visitor: print one name.
WORDS	(--)		List every word's name.
NEAREST	(-- a)		The lowest header above SOUGHT, while DATA-END looks.
(NEARER)	(x --)		DATA-END's visitor.
DATA-END	(xt -- a)		Where xt's data ends: the next header's PARENT cell, or HERE
CLONE	(xt "name" --)		A new word made from xt, which becomes its parent (Self-style).
NOOP	(--)		Do nothing.
SOURCE	(-- a n)		The text being read: its address and length.
DEFER	("name" --)		Make a deferred word: it runs the xt in its data cell (NOOP until IS).
(VECTOR)	(--)		A deferred word made only to lend its DOES> code to the kernel's WORD and FIND.
IS	(xt "name" --)	A	Aim a deferred word at xt.
DIGIT?	(c -- n f)		Character to digit value, and whether it is a digit in BASE.
SIGN?	(a n -- a n f)		Skip a leading '-' (if more follows) and say so.
>NUM	(u a n -- u a n)		Accumulate digits in BASE while they last.
(NUMBER?)	(a n -- n -1 | 0)		Convert a string (optional '-') in BASE.
BASE-PREFIX	(c -- base | 0)		The base a prefix character picks: $ 16, # 10, % 2; else 0.
NUMBER?	(c-addr -- n -1 | 0)		Convert a counted string; $ # % prefixes pick base 16, 10, 2.
MSG	(-- a)		Register $1A: the current error message (counted), 0 = none.
HALT	(--)		Stop the machine after this step (writes the STOP register).
SP!	(n --)		Set the data stack depth in bytes: 0 SP! empties it
ERRBUF	(-- c-addr)		$EF00: room for a message built at run time
PLACE	(a n c-addr --)		Store a n as a counted string
+PLACE	(a n c-addr --)		Append a n to one
ERROR	(c-addr --)		Raise an error with this message: MSG := it, then HALT.
(ABORT")	(f --)		Run-time of ABORT": raise the inline message if the flag is true.
ABORT"	(f "ccc<quote>" --)	I	Compiling: if the flag is true at run time, raise the error text.
(ABORT)	(--)		Empty the data, compile and context stacks; clear the error.
ABORT	(--)		(ABORT), then HALT.
UNKNOWN	(c-addr --)		Raise UNKNOWN WORD: name.
(REC-IN)	(c-addr wl -- xt -1 | c-addr 0)		Recognize a name in one wordlist.
REC-COMMON	(c-addr -- xt -1 | c-addr 0)		Recognizer: a COMMON word.
REC-COMPILER	(c-addr -- xt -1 | c-addr 0)		Recognizer: a COMPILER word.
REC-FORTH	(c-addr -- xt -1 | c-addr 0)		Recognizer: a FORTH word.

Word	Stack		What it does
`REC-NUMBER`	`( c-addr -- n -1 | c-addr 0 )`		Recognizer: a number.
`REC-STRING`	`( c-addr -- a n -1 | c-addr 0 )`		Recognizer: a "string" token, case kept, spaces and all.
`INTERPRETING-FRAME`	`( -- a )`		Context frame for mode 0: COMMON run, FORTH run, strings and numbers pushed.
`COMPILING-FRAME`	`( -- a )`		Context frame for modes 1-2: COMMON and COMPILER run, FORTH compiled, literals.
`MODE-FRAMES`	`( -- a )`		The context frame for each mode 0-4 (the tether re-aims them).
`ACTIVE-FRAME`	`( -- a )`		The context frame for the current mode.
`RECOGNIZE`	`( c-addr -- data action | c-addr 0 )`		Try the active frame's recognizers in order: the data and its action, or 0.
`INTERPRET`	`( -- )`		Read and act on every name in the source.
`EVALUATE`	`( a n -- )`		Interpret a string, then resume the current source.
`LINE`	`( -- a )`		Register $1C: the source line number, for error messages. (GRAPHICS' LINE shadows it.)
`TXT`	`( -- a )`		Register $1E: the next line of the loaded text.
`TXTEND`	`( -- a )`		Register $20: the end of the loaded text.
`REFILL`	`( -- f )`		Frame the next line of the loaded text as the source; false when none is left.
`QUIT`	`( -- )`		The outer loop: clean up after an error, then interpret line after line.
`QUIT-VECTOR`	`( -- a )`		Register $38: the word the machine enters to run text.
`ABORT-VECTOR`	`( -- a )`		Register $3A: the word the machine enters to clean up.
`FORGETTING`	`( a -- a )`		Deferred hook a MARKER runs before forgetting: modules chain their cleanup onto it.
`(UNAIM)`	`( w -- )`		MARKER's visitor: re-aim a deferred word that points into forgotten memory at NOOP.
`(FORGET)`	`( a -- )`		A marker's action: run the hooks, then restore HERE, LATEST and the wordlists.
`MARKER`	`( "name" -- )`		Make name: running it forgets name and everything made after it.
`ANEW`	`( "name" -- )`		MARKER, but first run an older marker of the same name (safe reloading).

NUMBERS

Word	Stack		What it does
`DECIMAL`	`( -- )`		Base 10.
`HEX`	`( -- )`		Base 16.
`U/MOD`	`( u1 u2 -- r q )`		Unsigned divide: remainder and quotient.
`DIGIT`	`( n -- c )`		Digit value → character.
`(U.)`	`( u -- )`		Print an unsigned number's digits alone, no space.
`(.)`	`( n -- )`		Print a signed number's digits alone, no space.
`U.`	`( u -- )`		Print an unsigned number in BASE, then a space.
`.`	`( n -- )`		Print a signed number in BASE, then a space (standard Forth).

TOOLS

Word	Stack		What it does
`FOUND`	`( -- a )`		Set by (IS-IT) when HEADER? finds its address.

Word	Stack		What it does
(IS-IT)	(x --)		Visitor for HEADER?.
HEADER?	(a -- f)		Is a the address of a word?
.NAME	(xt --)		Print a word's name (:NONAME if it has none).
FAR	(-- a)		SEE: the furthest forward branch seen.
.LIT	(a -- a')		SEE: print a literal (or [']) word).
.TEXT	(a -- a')		SEE: print an inline string.
.STR	(a -- a')		SEE: print an inline string as S" or .").
.BRANCH	(a xt -- a')		SEE: print a branch and its target.
.NESTED	(a -- a')		SEE: print a nested definition.
(CREATE)?	(a -- f)		SEE: is this CREATE's compiled form?
.CELL	(a -- a' f)		SEE: print one cell of a thread; f = the thread ended.
.THREAD	(a --)		SEE: print a thread up to its real end.
SEE-CODE	(xt --)		SEE a primitive.
.LINEAGE	(xt --)		Print a word's maker and its maker's makers.
PLAIN?	(xt -- f)		Was the word made by : or :NONAME?
SEE-COLON	(xt --)		SEE a colon word.
.VALUE	(n --)		Print a number, or ' word for an xt.
(CHILD)	(x --)		Visitor for CHILDREN.
CHILDREN	(xt --)		List the words xt made.
SEE-CREATE	(xt --)		SEE a data word: maker, contents, DOES> code.
SEERS	(-- a)		SEE's printers, one per KIND.
SEE	("name" --)		Decompile a word.
.S	(--)		Print the data stack, oldest first, without changing it.

GRAPHICS

Word	Stack		What it does
SPRITES	(-- a)		$F000: the 8 sprites' registers.
SPRITE	(n -- a)		Sprite n's registers: X Y PATTERN ON.
SPRITE-AT	(x y n --)		Move sprite n.
SHOW	(pattern n --)		Give sprite n a pattern and switch it on.
HIDE	(n --)		Switch sprite n off.
FRAMES	(-- a)		Register $24: frames shown so far.
VBL-VECTOR	(-- a)		Register $28: the xt the frame interrupt runs.
VBL	(--)		Deferred: the frame interrupt runs it 60 times a second.
POINTERS	(-- a)		$F048: the 4 pointer points.
(POINTER)	(n -- a)		Point n's registers: X Y DOWN.
POINTER	(n -- x y)		Where point n is, in screen pixels.
PRESSED?	(n -- f)		Is button / finger n down?
LX	(-- a)		LINE's working cell: current x.
LY	(-- a)		LINE's working cell: current y.
DX	(-- a)		LINE's working cell: distance across.
DY	(-- a)		LINE's working cell: minus the distance down.
SX	(-- a)		LINE's working cell: x step (1 or -1).
SY	(-- a)		LINE's working cell: y step (1 or -1).
ERR	(-- a)		LINE's working cell: the error term.

Word	Stack		What it does
HUE	(-- a)		LINE's working cell: its colour.
DIR	(n -- 1|-1)		The sign of a step.
LINE	(x0 y0 x1 y1 rgb --)		Draw a line (Bresenham, integers only).

MOTORS

Word	Stack		What it does
FIELD	(offset "name" --)		A word that adds its offset to an address
>ACT	(m -- a)		Field: the motor's job (an xt).
>FLAGS	(m -- a)		Field: RUN KILL BOUNCE WRAP ONCE bits.
>COUNT	(m -- a)		Field: frames left on its timer.
>CD	(m -- a)		Field: the flags to toggle (byte), then the reload count (byte).
>BEFORE	(m -- a)		Field: hook run before the job.
>AFTER	(m -- a)		Field: hook run after the job.
>A	(m -- a)		Field: operand A (what it works on).
>B	(m -- a)		Field: operand B.
>C	(m -- a)		Field: operand C.
>D	(m -- a)		Field: operand D.
F-RUN	(-- bit)		Flag: the motor works.
F-KILL	(-- bit)		Flag: remove it at the end of the frame.
F-BOUNCE	(-- bit)		Flag: LIMIT negates the velocity at D.
F-WRAP	(-- bit)		Flag: LIMIT wraps to the other limit.
F-ONCE	(-- bit)		Flag: PATH stops at the end of its table.
ME	(-- a)		Register $26: the motor at work.
MOTORS	(-- a)		The first motor; they run in the order made.
FLAG+	(bits m --)		Set flag bits.
FLAG-	(bits m --)		Clear flag bits.
FLAG?	(bits m -- f)		Test flag bits.
START	(m --)		Run the motor.
PAUSE	(m --)		Pause the motor.
KILL	(m --)		Remove the motor at the end of this frame.
BOUNCES	(m --)		Make a LIMITER bounce.
WRAPS	(m --)		Make a LIMITER wrap.
ONCE	(m --)		Make a PATH run once.
TIMER	(frames toggle reload m --)		Give a motor a timer: after n frames toggle these flags, then reload (0 = once).
BEFORE	(xt m --)		Hang a hook (m --) before the job.
AFTER	(xt m --)		Hang a hook (m --) after the job.
MINE	(-- m)		The motor at work (from ME).
ADD	(--)		Job: [A] += [B].
LANDS	(v --)		LIMIT's landing: store v, and bounce if asked.
LIMIT	(--)		Job: keep [A] within B..C (clamp, wrap or bounce).
PATH-STEP	(--)		Job: every D frames, [A] := the next table entry.
MOVE-STEP	(--)		Job: copy C bytes from B to A (B = 0: zero-fill).
CODE-STEP	(--)		Job: run the word in A with the motor on the stack.
MOTOR	(a b c d act -- m)		Lay a motor record, link it last, running.
ADDER	(dst src -- m)		Motor: every frame, [dst] += [src].

Word	Stack		What it does
LIMITER	(a min max velocity -- m)		Motor: keep [a] within min..max.
PATH	(a table every -- m)		Motor: step [a] through a table every n frames.
MOVER	(dst src n -- m)		Motor: copy n bytes every frame.
CODER	(xt -- m)		Motor: run xt (m --) every frame.
HOOK	(m cell -- m)		Run a hook cell's xt with the motor, if set.
TICK	(m -- m)		Count a motor's timer down; at 0 toggle and reload.
WORK	(m --)		Run one motor: hooks, job, timer.
RUN-MOTORS	(--)		Run every motor once (the VBL job); unlink killed ones.
REACH	(m -- a)		The highest address a motor lives at or uses.
FORGET-MOTORS	(a -- a)		FORGETTING hook: unlink motors a MARKER forgets.

TIME

Word	Stack		What it does
SECONDS	(-- a)		$F060: seconds (byte).
MINUTES	(-- a)		$F061: minutes (byte).
HOURS	(-- a)		$F062: hours 0-23 (byte).
DAY	(-- a)		$F063: day 1-31 (byte).
MONTH	(-- a)		$F064: month 1-12 (byte).
YEAR	(-- a)		$F066: year (cell).
TIME&DATE	(-- sec min hour day month year)		Read the clock chip.
NOW	(-- sec minute-of-day)		Seconds, and minutes since midnight (0-1439).
MS	(n --)		Sleep n milliseconds (in frames).
EVERY	(xt n -- m)		Motor: run xt (m --) every n frames.
LATER	(xt n -- m)		Motor: run xt (m --) once, n frames from now.
DUE?	(m -- f)		Has the alarm's time come?
ALARM-STEP	(--)		Job: ring once when due.
ALARM	(xt h m s -- m)		Motor: run xt at h:m:s (tomorrow, if already past).

SOUND

Word	Stack		What it does
VOICES	(-- a)		$F200: the 4 voices' registers.
VOICE	(n -- a)		Voice n's registers.
SQUARE	(-- wave)		Waveform 1.
SAWTOOTH	(-- wave)		Waveform 2.
TRIANGLE	(-- wave)		Waveform 3.
SINE	(-- wave)		Waveform 4.
NOISE	(-- wave)		Waveform 5.
TONE	(freq vol wave n --)		Sound voice n until QUIET.
QUIET	(n --)		Silence voice n.
PLAY	(a len rate vol n --)		Play an 8-bit sample from RAM on voice n.

META

Word	Stack		What it does
MACHINE	(-- a)		The pretend machine at work.
LEVEL	(-- a)		How many META levels are running.

Word	Stack		What it does
MACHINES	(-- a)		Four pretend machines, 72 bytes each.
PIP	(-- a)		The pretend IP.
PRP	(-- a)		The pretend return stack depth.
THEIRS	(-- a)		What the program being run sees in MACHINE.
MARK	(-- a)		The real return stack depth when this level began.
PRS	(-- a)		The pretend return stack (32 cells).
>PR	(x --)		Push onto the pretend return stack.
PR>	(-- x)		Pop the pretend return stack.
CELL>	(-- x)		Read the next cell of the thread.
UNSELF	(-- a)		A one-cell thread: where pretend DOES> code returns.
ENTER-COLON	(xt --)		DOCOL, in Forth.
ENTER-DOES	(xt --)		DODOES, in Forth.
FOR-REAL	(xt --)		Run a primitive for real, with the program's own MACHINE.
DISPATCH	(xt --)		Do one xt, as the kernel's EXECUTE would.
STEP	(--)		One step of the pretend machine.
RP-NOW	(-- n)		The real return stack depth.
GIVE-BACK	(rp -- rp)		Free machines left behind by an error.
MACHINE-ON	(xt rp -- xt)		Take the next pretend machine.
MACHINE-OFF	(--)		Back to the level above.
META	(xt --)		Run xt on the Forth-in-Forth interpreter.
.STEP	(--)		TRACE: print the next step, indented, with the stack.
TRACE	(xt --)		META, printing every step with the stack.

SERIAL

Word	Stack		What it does
TX-HEAD	(-- a)		$F300: where the wire takes the next byte out.
TX-TAIL	(-- a)		$F301: where SEND puts the next byte.
RX-HEAD	(-- a)		$F302: where RECV takes the next byte.
RX-TAIL	(-- a)		$F303: where the wire puts the next byte in.
TX-RING	(-- a)		$F400: 256 bytes going out.
RX-RING	(-- a)		$F500: 256 bytes coming in.
SEND	(c --)		A byte down the cord (waits while TX is full).
RX?	(-- f)		Is a byte waiting?
RECV	(-- c)		A byte up the cord (waits for one).
RX-ACCEPT	(a n -- n')		ACCEPT from the UART: a line, echoed back down the cord.
SERVE	(--)		QUIT on the UART: read lines from the cord, interpret, answer ok.

TETHER

Word	Stack		What it does
SEND16	(n --)		A cell down the cord, low byte first.
RECV16	(-- n)		A cell up the cord, low byte first.
SENDS	(a n --)		N bytes down the cord.
CMD	(a n c --)		Send a monitor command: byte c, address a, length n.
TC!	(c a --)		Store a byte in the target (W).
T!	(n a --)		Store a cell in the target (W).

Word	Stack		What it does
TC@	(a -- c)		Fetch a byte from the target (R).
T@	(a -- n)		Fetch a cell from the target (R).
TMOVE	(a ta n --)		Copy n of our bytes into the target, 240 at a time.
RECV+	(c-addr --)		Append a byte from the cord to a counted string.
(TCALL)	(xt -- f)		Run a target word (X); show its output; f = it failed (message in ERRBUF).
TCALL	(xt --)		Run a target word; its error becomes ours.
>T	(n --)		Push onto the target's data stack.
T>	(-- n)		Pop the target's data stack.
.TS	(--)		Print the target's data stack.
RECVS	(a n --)		N bytes from the cord into a.
TREAD	(ta a n --)		Copy n target bytes into ours, 240 at a time.
TDP	(-- a)		The target's HERE, kept here while we build.
TLINK	(-- a)		The target's newest FORTH header.
TLAST	(-- a)		The target's LATEST.
THERE	(-- a)		The target's HERE.
T,	(x --)		Compile a cell into the target.
TREGS	(-- a)		The target's registers $04-$37, as last read.
(PULL)	(--)		Read the target's registers into TREGS; take its HERE and FORTH head.
CONNECT	(--)		The first time: learn where the target's dictionary is.
TARGET-WORDS	(-- a)		Wordlist: shadows of target words.
TARGET-COMPILER	(-- a)		Wordlist: mode 3's own words (; ['] S" …).
TARGET-INTERPRETER	(-- a)		Wordlist: the prompt's words in target mode.
H,	(x --)		A cell into our own dictionary, whatever space is being built.
MAKE-SHADOW	(txt imm? c-addr --)		Make a shadow: a host word naming a target xt.
SHADOW	(txt -- xt | 0)		The newest shadow of a target xt (0: none).
T-HEADER	(tparent kind c-addr -- txt)		Build a header at THERE, and its shadow here.
T-LINK	(txt --)		Link a finished target word into its FORTH list; it becomes LATEST there.
TBUF	(-- a)		258 bytes: a target stack or header on its way back.
TOLD	(-- a)		Headers at or above it are new to us.
THEAD	(h -- h)		Read a target header's LINK LEN NAME into TBUF.
TURN	(old --)		Reverse the shadows made since the list head was old.
NEWS	(h imm? --)		Shadows for a target list's headers at or above TOLD.
CATCH-UP	(--)		Shadows for every new header in the target's three lists; update TLAST.
PRUNE	(ta --)		Drop every shadow of a target word at or above ta.
TRUN	(i*x txt -- j*x)		Run a target word as if it were ours: our stack and the rest of our line go there, its stack comes back.
REC-TARGET	(c-addr -- txt -1 | c-addr 0)		Recognizer: a target word (by its shadow).
KERNEL-HEAD	(-- xt)		The newest kernel header (the first FORTH header below KEND).
KERNEL-WORDS	(-- a)		A wordlist: the kernel's own words, the same on both machines.
REC-KERNEL	(c-addr -- xt -1 | c-addr 0)		Recognizer: a kernel word.
REC-TCOMPILER	(c-addr -- xt -1 | c-addr 0)		Recognizer: mode 3's own words (TARGET-COMPILER).

Word	Stack		What it does
REC-TIMMEDIATE	(c-addr -- txt -1 | c-addr 0)		Recognizer: a target word that acts while compiling.
TARGET?	(-- f)		Are we in target mode?
REC-TINTERP	(c-addr -- xt -1 | c-addr 0)		Recognizer: the prompt's target-mode words (TARGET-INTERPRETER).
REC-THERE	(c-addr -- txt -1 | c-addr 0)		Recognizer, target mode: anything the target has.
T'	("name" -- txt)		A target xt by name (a target word, or a kernel word).
TFIND	(a n -- txt | 0)		A target word by name (0: none).
REMOTE	(txt --)		Compile a call that runs a target word via TRUN.
NOT-THERE	(a n --)		Raise: name is not on the target yet.
T-XT	(a n -- txt | 0)		A target word or kernel word by name (0: none).
T-NAMED	(a n -- txt)		T-XT, or raise: not on the target yet.
TRUN-NAMED	(i*x a n -- j*x)		Run the target's word of that name.
>T-XT	(xt -- txt | 0)		The target's namesake of one of our words.
T-OPEN	(parent kind c-addr --)		The target space's OPEN: a colon header there, frames here.
TARGET-SPACE	(-- a)		The space table: THERE T, T! T-OPEN (HERE , PATCH OPEN).
TARGET	(--)		Build new definitions in the target.
HOST	(--)		Build new definitions here again.
T-CREATE	(tparent "name" --)		Make a data word in the target.
NOT-YET	("name" --)		Make a word that refuses: it would lay host xts in the target.
FORWARD	("name" --)		Make a word that runs the target's word of the same name.
T-IMMEDIATE?	(txt -- f)		Does the target word act while compiling there?
MARK-IMMEDIATE	(--)		Mark the target's newest word's shadow as acting while compiling.
(T,NAMED)	(a n --)		Compile the target's word of that name into the target.
T-COMPILE	("name" --)	I	In a transition word: compile the named target word, looked up when it runs.
TS,	(a n --)		Compile a counted string into the target.
TSLITERAL	(a n --)		SLITERAL, in the target.
TETHERED-INTERPRETING	(-- a)		Context frame for mode 0 with a tether.
TETHERED-COMPILING	(-- a)		Context frame for modes 1-2 with a tether.
TARGET-COMPILING	(-- a)		Context frame for modes 3-4: building a target word.
(UNSHADOW)	(a wl -- a)		Drop the shadows a marker forgets, from one wordlist.
FORGET-SHADOWS	(a -- a)		FORGETTING hook: shadows go with a MARKER.
CUT	(--)		Cut the cord: the target's monitor enters SERVE; it serves its own Forth.
TERM	(--)		Be the target's terminal: its bytes shown, our keys sent. ESC leaves.
SYNCS	(-- a)		How many sync nonces have been sent.
WAITED	(-- a)		Frames SYNC has waited without a byte.
SYNC-WANT	(-- a)		The answer SYNC waits for: EOT Y nonce.
SYNC-WIN	(-- a)		The last 4 bytes up the cord.
(ASK)	(--)		Send EOT and Y with a new nonce.
SYNC	(--)		The handshake: EOT, then read until the monitor's EOT Y nonce (ask again after 30 quiet frames).

Word	Stack		What it does
`TRUE?`	`( xt -- f )`		Is this shadow still true on the target (below its HERE, same name)?
`SURVIVOR`	`( -- txt | 0 )`		Our newest shadow still true on the target (0: none).
`TETHER`	`( -- )`		Take the cord back: SYNC with the target's monitor, then bring the shadows up to date.

Target-mode words (TETHER's own wordlists)

These do not appear in the host's three wordlists. The tethered context frames search them first while you build in the target.

TARGET-COMPILER: used while a target word is being compiled (modes 3 and 4). Each one does what BOOT's word of the same name does, but lays the **target's** xts:

Word	Stack	What it does there
`;`	`( -- )`	Lay EXIT, close both frames, link the word into the target's FORTH list, write its HERE
`[']`	`( "name" -- )`	Compile LIT and the target xt of name
`S"`	`( "ccc<quote>" -- )`	Compile the target's `(S")` and the string
`."`	`( "ccc<quote>" -- )`	As S", then the target's `TYPE`
`ABORT"`	`( "ccc<quote>" -- )`	Compile the target's `(ABORT")` and the message
`CREATE`	`( "name" -- )`	Compile the target's `(CREATE)`, with the maker (or SELF in DOES> code)
`DOES>`	`( -- )`	Compile the target's `(DOES>)`; switch to mode 4
`POSTPONE`	`( "name" -- )`	Postpone a target word
`OF`	`( C: -- orig )`	CASE's OF, with the target's OVER = DROP

TARGET-INTERPRETER: the prompt's words in target mode. These build *there* directly, with no round trip, or run the target's own word:

Word	Stack	What it does
`, C,`	`( x -- )`	A cell / byte at the target's HERE
`HERE`	`( -- a )`	The target's HERE (THERE)
`ALLOT`	`( n -- )`	Reserve n bytes there
`'`	`( "name" -- txt )`	A target xt (T')
`CREATE VARIABLE`	`( "name" -- )`	A data word there
`]`	`( -- )`	Ours: it pops *our* context stack
`CONSTANT DEFER IS`	as BOOT's	Forwarded: the target's own word runs there, parsing from our line
`IMMEDIATE ALWAYS`	`( -- )`	Run the target's own, and mark its shadow as acting while compiling

Switched off in the source

Counted loops (BOOT §13: delete the leading `\` to enable):

Word	Stack		What it does
`DO`	`( limit start -- )`	I	Begin a counted loop, I = start … limit-1
`?DO`	`( limit start -- )`	I	As DO, but skip the loop if limit = start
`LOOP`	`( -- )`	I	Step I by 1; loop until it reaches limit
`+LOOP`	`( n -- )`	I	Step I by n; stop when it crosses the limit
`I J`	`( -- n )`		The inner / outer loop index
`LEAVE`	`( -- )`		Leave the loop now
`UNLOOP`	`( -- )`		Drop the loop's return-stack cells (before EXIT)

Word	Stack		What it does
`(DO)` `(?DO)` `(LOOP)` `(+LOOP)`			Their run-time parts

Speed primitives (kernel APPENDIX): `NEGATE INVERT OR XOR OVER ROT NIP 2DUP 2DROP R@ 1+ 1- 0= 0< = < U< UPC SKIP PARSE`. Each exists as Forth in BOOT; uncomment its line in `speedPrimitives()` and delete the Forth definition to make it machine code.

The trap messages (ROM at $EC00)

#	Message	#	Message
0	DATA STACK OVERFLOW	10	; outside colon definition
1	DATA STACK UNDERFLOW	11	; with an unclosed IF, BEGIN or CASE
2	RETURN STACK OVERFLOW	12	; of another space: that space has its own
3	RETURN STACK UNDERFLOW	13	' word not found
4	COMPILE STACK OVERFLOW	14	WAIT: nothing to wake it
5	CONTROL STRUCTURE MISMATCH	15	MAX STEPS EXCEEDED (runaway loop?)
6	CONTEXT STACK OVERFLOW	16	BREAK
7	CONTEXT STACK UNDERFLOW	17	UM/MOD OVERFLOW
8	a nested word is built where the word around it is	18	] without [
9	DICTIONARY FULL	19	UNKNOWN WORD

PART IV · The Quick Reference

```
================================================================================
  F O R T H S C R I P T   (Forth 2x2)                      QUICK REFERENCE
  the ITCH system  ·  16-bit  ·  64K bus  ·  indirect threaded  ·  1985 style
================================================================================

--------------------------------------------------------------------------------
 0. THE LANGUAGE IN ONE SCREEN
--------------------------------------------------------------------------------

  Everything is a word, separated by spaces. Words take their arguments
  from the data stack and leave their results there.

      2 3 + .                    prints 5
      : SQUARE ( n -- n*n ) DUP * ;
      7 SQUARE .                 prints 49

  Tokens, in the order the interpreter tries them:
      a word            run it (interpreting) or compile it (compiling)
      "a string"        push a n, case and spaces kept   ( "Hi there" TYPE )
      a number          123  -45  $FF (hex)  #99 (decimal)  %101 (binary)

  Comments:   \ to end of line        ( up to the closing paren )

  Case:       names are found in any case: dup = DUP
  Flags:      0 is false, -1 is true (all bits set); any non-zero tests true
  Numbers:    .  prints a number and then a space, as in standard Forth:
              1 2 3 . . .  shows 3 2 1.  (.) prints the digits alone

--------------------------------------------------------------------------------
 1. STACK NOTATION
--------------------------------------------------------------------------------

  ( before -- after )   top of stack on the right
  n signed   u unsigned   f flag   c char   a address   xt a word
  c-addr counted string [len][chars]    a n  address and length
  "name" parsed from the input     C: compile stack     R: return stack

--------------------------------------------------------------------------------
 2. STACK
--------------------------------------------------------------------------------

  DUP      x -- x x          copy top            DROP     x --          discard
  SWAP     a b -- b a        exchange            OVER     a b -- a b a  copy 2nd
  ROT      a b c -- b c a    third to top        NIP      a b -- b      drop 2nd
  2DUP     a b -- a b a b                        2DROP    a b --
  ?DUP     x -- x x | 0      dup if non-zero     DEPTH    -- n          cells
  >R       x --  R: -- x     to return stack     R>       -- x  R: x -- from it
  R@       -- x              copy top of R       .S       --  print stack
  SP!      n --              set depth (bytes)   0 SP! empties the stack

--------------------------------------------------------------------------------
 3. ARITHMETIC & LOGIC                          (16-bit, wraps silently)
--------------------------------------------------------------------------------

  +  -  *        a b -- n                /      a b -- q    toward zero
  MOD            a b -- r                1+ 1-  n -- n
  NEGATE ABS     n -- n                  MIN MAX a b -- n    signed
  U/MOD          u1 u2 -- rem quot       UM/MOD  lo hi u -- rem quot
  AND OR XOR     a b -- x                INVERT  x -- ~x
  =  <>  <  >  <=  >=        a b -- f    signed
  U<  U>                     u u -- f    unsigned
  0=  0<>  0<  0>            n -- f
  WITHIN         n lo hi -- f            lo <= n < hi

--------------------------------------------------------------------------------
 4. MEMORY
--------------------------------------------------------------------------------

  @   a -- x     fetch cell          !   x a --     store cell
  C@  a -- c     fetch byte          C!  c a --     store byte
  +!  n a --     add to cell         MOVE from to n --   copy bytes
  FILL a n c --  fill bytes          HERE -- a      next free byte
  ,   x --       compile a cell      C,  c --       compile a byte
  ALLOT n --     reserve bytes       PAD -- a       scratch area

--------------------------------------------------------------------------------
 5. CONTROL FLOW                                    (inside definitions)
--------------------------------------------------------------------------------

  f IF ... THEN
```

```
  f IF ... ELSE ... THEN
  BEGIN ... f UNTIL                  loop until true
  BEGIN ... AGAIN                    forever
  BEGIN ... f WHILE ... REPEAT       test in the middle
  BEGIN ... f WHILE ... f WHILE ... REPEAT THEN     two exits
  x CASE  1 OF ... ENDOF  2 OF ... ENDOF  default ( x still there )  ENDCASE
  RECURSE                            call the word being defined
  EXIT                               return now

  : SIGN ( n -- ) DUP 0< IF DROP ." negative" ELSE
      0= IF ." zero" ELSE ." positive" THEN THEN ;
  : COUNTDOWN ( n -- ) BEGIN DUP . 1- DUP 0= UNTIL DROP ;

  DO LOOP etc. exist but are switched off: see BOOT section 13.

--------------------------------------------------------------------------------
 6. DEFINING WORDS
--------------------------------------------------------------------------------

  : name ... ;            a colon word              :NONAME ... ;  -- xt
  n CONSTANT name         name pushes n             VARIABLE name  a cell, 0
  CREATE name             name pushes its address   DEFER name     aim with IS
  xt IS name              aim a deferred word       CLONE name     xt "name" --
  IMMEDIATE               newest word acts while compiling
  ALWAYS                  newest word acts the same in every mode
  [ ... ]                 interpret inside a definition
  LITERAL   n --  compile n            POSTPONE name  compile its action
  ' name  -- xt           ['] name   (compiling)    EXECUTE  xt --

  DOES>: words that make words
      : ARRAY ( n "name" -- ) CREATE 2 * ALLOT DOES> SWAP 2 * + ;
      5 ARRAY PTS   42 3 PTS !   3 PTS @ .           prints 42

  Nesting: a : inside a : builds a second word, jumped over by the first
      : OUTER 1 : INNER 2 ; INNER + ;   OUTER .      prints 3

--------------------------------------------------------------------------------
 7. LINEAGE                                  every word knows its parent
--------------------------------------------------------------------------------

  PARENT     xt -- xt'     the word that made it (0: the machine)
  IS-A?      xt anc -- f   is anc on its family line?
  CHILDREN   xt --         list the words it made
  SELF       -- xt         the data word whose DOES> code is running
  SEE PTS    CREATE PTS ( made by ARRAY, holds 0 ) DOES> SWAP 2 * + ;

--------------------------------------------------------------------------------
 8. TEXT & STRINGS
--------------------------------------------------------------------------------

  EMIT  c --        CR  --      SPACE  --       PAGE  --  clear the terminal
  TYPE  a n --      COUNT  c-addr -- a n        . n --  U. u --  (+ a space)
  (.)  n --         (U.)  u --  digits alone, no space: for laying out by hand
  ." text"          print (compiled, or now)    .( text)  print now, always
  S" text"  -- a n  a string                    "text"    -- a n  the same
  PLACE  a n c-addr --      +PLACE  a n c-addr --  append
  S=  a1 a2 n -- f  equal, any case             UPPER  a n --
  HEX  DECIMAL      BASE  -- a                  /STRING  a n k -- a' n'

--------------------------------------------------------------------------------
 9. INPUT
--------------------------------------------------------------------------------

  KEY  -- c         wait for a key              KEY?  -- f
  KEY-DOWN?  c -- f   held right now (games)    ACCEPT  a n -- n'  a line
  WORD  c -- c-addr   parse a name to PAD       PARSE  c -- a n
  Keys: ASCII, 8 bksp 9 tab 13 enter 27 esc 28 left 29 right 30 up 31 down

--------------------------------------------------------------------------------
 10. THE INTERPRETER & DICTIONARY
--------------------------------------------------------------------------------

  WORDS  --          list all words             SEE name  decompile
  FIND  c-addr -- xt 1 | xt -1 | c-addr 0       FIND-NAME  a n -- xt|0
  EVALUATE  a n --   interpret a string         INTERPRET  QUIT  REFILL
  MARKER name        name forgets itself and all after it
  ANEW name          MARKER, after running an older one (for reloading)
  COMMON-WORDS COMPILER-WORDS FORTH-WORDS  -- a   the three wordlists
  Wordlists decide behaviour: COMPILER = immediate, COMMON = always.

--------------------------------------------------------------------------------
 11. ERRORS
--------------------------------------------------------------------------------

  f ABORT" text"     error if f is true         c-addr ERROR   raise it
```

```
  ABORT              empty the stacks            HALT          stop now
  An error stops the machine with the stacks as they were. The next line
  cleans up first. MSG -- a holds the current message (0 = none).

--------------------------------------------------------------------------------
 12. GRAPHICS                     320 x 200, colours $RGB ($F80 = orange)
--------------------------------------------------------------------------------

  CLS  --                      clear to black
  PLOT  x y rgb --             a pixel           POINT  x y -- rgb
  LINE  x0 y0 x1 y1 rgb --     a line
  SPLOT  x y rgb pat --        a pixel of sprite pattern 0-7 (0 = clear)
  pat n SHOW   n HIDE           8 sprites, 64x64, sprite 0 on top
  x y n SPRITE-AT               move one
  n POINTER  -- x y    n PRESSED?  -- f       mouse = 0, fingers 0-3
  FRAMES  -- a                  frames shown (60 a second)
  ' word IS VBL                 run word every frame (MOTORS aims it at
                                RUN-MOTORS: re-aiming stops the motors)

--------------------------------------------------------------------------------
 13. MOTORS                  workers run every frame, wired through RAM
--------------------------------------------------------------------------------

  dst src ADDER  -- m           [dst] += [src]
  a min max vel LIMITER  -- m   keep [a] in range   ( m BOUNCES / m WRAPS )
  a table every PATH  -- m      step [a] through a table  ( m ONCE )
  dst src n MOVER  -- m         copy n bytes every frame
  xt CODER  -- m                run xt ( m -- ) every frame
  m START  m PAUSE  m KILL      n toggle reload m TIMER
  xt m BEFORE   xt m AFTER      hooks ( m -- )

  VARIABLE X
  : ARROWS ( m -- ) DROP  28 KEY-DOWN? IF -2 X +! THEN
                          29 KEY-DOWN? IF  2 X +! THEN ;
  ' ARROWS CODER DROP     X 0 288 0 LIMITER DROP

--------------------------------------------------------------------------------
 14. TIME & SOUND
--------------------------------------------------------------------------------

  TIME&DATE  -- s m h day month year          NOW  -- s minute-of-day
  n MS                     sleep n milliseconds
  xt n EVERY  -- m         every n frames      xt n LATER -- m  once, later
  xt h m s ALARM  -- m     at a time of day
  freq vol wave n TONE     voice n (0-3)       n QUIET
  SQUARE SAWTOOTH TRIANGLE SINE NOISE  -- wave
  a len rate vol n PLAY    an 8-bit sample from RAM

--------------------------------------------------------------------------------
 15. META                                      Forth running Forth
--------------------------------------------------------------------------------

  xt META         run xt on an interpreter written in Forth (4 levels deep)
  xt TRACE        the same, printing every step with the stack
      ' 1+ 41 SWAP META .                          prints 42

--------------------------------------------------------------------------------
 16. THE CORD: TWO MACHINES                    SERIAL + TETHER modules
--------------------------------------------------------------------------------

  SEND  c --       RECV  -- c      RX?  -- f  one byte each way
  x a T!   a T@ -- x   c a TC!   a TC@ -- c   the target's memory
  a ta n TMOVE           ta a n TREAD          blocks
  xt TCALL               run a target word     n >T   T> -- n   .TS

  TARGET  ...  HOST      build in the target, then here again
      TARGET : SQ DUP * ; HOST   5 SQ .          prints 25  (run THERE)
  T' name  -- txt        THERE  -- a           the target's xt, HERE
  CUT                    let go: the target serves its own Forth (SERVE)
  TERM                   be its terminal (ESC leaves)
  TETHER                 take the cord back; learn its new words
                         (a handshake: EOT, then a nonce the target echoes)

  A full session:
      TARGET  <BOOT NUMBERS TOOLS META SERIAL, as text>  HOST
      CUT  TERM
          : CUBE DUP DUP * * ;           <- typed to the target itself
          <ESC>
      TETHER  3 CUBE .                   prints 27, computed there

--------------------------------------------------------------------------------
 17. MEMORY MAP                                          (hex addresses)
--------------------------------------------------------------------------------

  0000 registers   0040 TIB   0440 PAD   04C0 data stack   05C0 return stack
```

```
  06C0 compile stack   07C0 context stack   0800 dictionary   8000 text
  EC00 ROM messages   EF00 ERRBUF   F000 sprites   F040 keys   F048 pointer
  F060 clock   F080 held keys   F100 key ring   F200 voices   F300 UART
  F400 TX   F500 RX   F600 terminal   F700 terminal ring

  Header:  PARENT(2) | LINK(2) LEN(1) NAME KIND(1) body
  KIND:    0 CODE  1 COLON  2 CREATED (ACTION cell, then data)

================================================================================
                        "two machines, one language"
================================================================================
```